IUT Saint-Dié-des-Vosges

UNIVERSITÉ DE LORRAINE

Algorithmes de segmentation
TP3

Hugo Rositi*

20 octobre 2026

---

1 Renforcer les contours

— Réaliser une détection de contours (avec l'opérateur de Canny par exemple).
— Tester la transformée de Hough en lignes pour renforcer les contours.
Les fonctions `cv.Canny()` et `cv.HoughLines()` vous seront utiles.

2 Segmentation par ligne de partage des eaux

1. Mettre en place une segmentation d'images par watershed (ligne de partage des eaux).
2. Vous améliorerez votre chaîne de traitement avec l'utilisation de marqueurs.

Pensez à aller voir la document de la fonction `cv.watershed()`. Pour rappel, les pages suivantes peuvent vous être utiles :
— Présentation de l'algorithme
— Watershed, wikipedia

3 Détection de visages

Dans cette section, nous allons voir comment détecter les visages à l'aide d'un classificateur en cascade dans OpenCV Python. La détection des visages revêt une grande importance dans différents domaines du monde d'aujourd'hui. C'est une étape importante dans plusieurs applications, la reconnaissance faciale (également utilisée comme biométrie), la photographie (pour l'autofocus sur le visage), l'analyse du visage (âge, sexe, reconnaissance des émotions), la vidéosurveillance, *etc.*

L'un des algorithmes populaires pour la détection faciale est « haarcascade ». Il s'agit d'un algorithme moins coûteux en termes de calcul, rapide et offrant une grande précision.

Nous allons implémenter la méthode proposée par Viola and Jones [1].

Cela fonctionne en quatre étapes :

*hugo.rositi@univ-lorraine.fr

1. RP. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition. CVPR 2001, Kauai, HI, USA, 2001, pp. I-I, doi : 10.1109/CVPR.2001.990517.

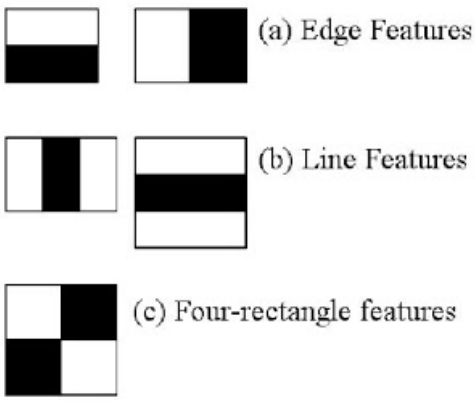


FIGURE 1 – Features retenus pour la détection de visage par Viola and Jones

1. Sélection des caractéristiques Haar : une caractéristique de type Haar se compose de régions sombres et de régions claires. Il produit une valeur unique en prenant la différence entre la somme des intensités des régions sombres et la somme des intensités des régions claires. Il s'agit d'extraire les éléments utiles nécessaires à l'identification d'un objet. Les "features" proposées par Viola and Jones sont visibles en figure 1
2. Création d'images intégrales : Un pixel donné dans l'image intégrale est la somme de tous les pixels à gauche et de tous les pixels au-dessus. Étant donné que le processus d'extraction des caractéristiques de type Haar implique le calcul de la différence entre les régions rectangulaires sombres et claires, l'introduction des images intégrales réduit considérablement le temps nécessaire pour accomplir cette tâche.
3. Formation AdaBoost : cet algorithme sélectionne les meilleures "features" parmi tous les features détectés. Il combine plusieurs « classificateurs faibles » (meilleures fonctionnalités) en un seul « classificateur fort ». Le « classificateur fort » généré est essentiellement la combinaison linéaire de tous les « classificateurs faibles ».
4. Classificateur en cascade : il s'agit d'une méthode permettant de combiner des classificateurs de plus en plus complexes comme AdaBoost dans une cascade qui permet d'éliminer rapidement les entrées négatives (non-visage, si on cherche des visages) tout en dépensant plus de calculs sur des régions semblables à des visages prometteuses ou positives. Cela réduit considérablement le temps de calcul et rend le processus plus efficace.

OpenCV est livré avec de nombreux classificateurs pré-entraînés. Ces fichiers XML peuvent être chargés par la méthode `cv2.cascadeClassifier()`. Ici, nous allons utiliser `haarcascade_frontalface_de` pour détecter les visages.

Voici un algorithme possible :

1. Charger l'image et la convertir en niveaux de gris.
2. Charger le fichier XML du classifieur Haar.
3. Détecter les visages.
4. Afficher un rectangle autour de chaque face

La méthode `cv2::CascadeClassifier::detectMultiScale` renvoie les boites englobantes des faces détectées (c'est-à-dire x, y, w, h). Il lui faut deux paramètres, à savoir scaleF actor et minN eighbors. ScaleF actor détermine le facteur d'augmentation de la taille de la fenêtre qui commence initialement à la taille « minSize », et après avoir testé toutes les fenêtres de cette taille, la fenêtre est agrandie par « scaleFactor » et la taille de la fenêtre augmente jusqu'à « maxSize ». Si le « scaleFactor » est grand (par exemple, 2,0), il y aura moins d'étapes,



FIGURE 2 – Détection de visages

donc la détection sera plus rapide, mais nous pourrions manquer des objets dont la taille se situe entre deux échelles testées. (le facteur d'échelle par défaut est 1,3). Plus les valeurs de « minNeighbors » sont élevées, moins il y aura de faux positifs et moins d'erreurs seront en termes de fausse détection de visages. Cependant, il est également possible que des traces floues du visage manquent.